■ ASVANNA (■■■■■■■■)

The Zero-Waste Marketplace: Guided by Real-Time Data from Seed to Harvest Distribution

Comprehensive System Architecture, Deep Codebase Reference, Group 15 Role Allocations & QA Manual

---

Academic Institution:	Division of Information Technology, Institute of Technology, University of Moratuwa (ITUM)
Academic Program:	National Diploma in Information Technology (NDIT) — 2025 / 2026
Project Supervisor:	Mrs. Uthpala Athukorala (Division of Information Technology, ITUM)
Target Geographic Pilot:	Bandarawela Agrarian Services Division, Badulla District, Sri Lanka
Target Agricultural Sector:	Upcountry Perishable Vegetables (Leeks, Cabbage, Carrot, Beetroot, Potato, Knol Khol, Tomato, Bell Pepper)

Group 15 — Team Members & Assigned Modules:

#	Student Name	Student ID	Assigned Engineering Module & Responsibility
1	W. N. A. Wedikkara	23IT0544	System Architecture, Auth (JWT/RBAC), PostgreSQL Schema, Predictive Risk Engine
2	K. A. H. I. Lakshitha	23IT0503	Smart Crop Recommendation Engine, 4-Factor Weighted Agro-Suitability Scorer
3	G. W. T. Jayampathi	23IT0487	Divisional Officer Portal, Proxy Data Entry, Trilingual Localization, Broadcast Alerts
4	R. R. L. Geeganage	23IT0476	Farmer Mobile App (Flutter), GPS Field Logging, Offline Storage Queue & Sync
5	K. H. M. Dewanga	23IT0467	Phase 2 Geo-Fenced 5 km Surplus Marketplace, Haversine Engine & Negotiation State Machine

Manual Guide: This document is crafted for non-technical stakeholders (farmers, agrarian officers) and technical academic evaluators. It contains plain English, Sinhala & Singlish explanations, system diagrams, and file-by-file code breakdowns.

1. Plain-Language Executive Summary (For Non-IT Persons & Farmers)

In Sri Lanka's central hill country (Bandarawela, Welimada, Nuwara Eliya), farming families lose an estimated **30% to 40% of their annual crop yield** (equivalent to over Rs. 180 Billion in national waste). The root cause is **'Trend Planting' (■■■ ■■■■■)**. When leeks or cabbage reach high prices at Dambulla or Manning Market, hundreds of farmers simultaneously plant the same vegetable. Months later, thousands of tons reach harvest on the exact same week, causing disastrous market surpluses and catastrophic price collapse.

■■ Sinhala & Singlish Simple Explanation (■■■■ ■■■■■■■ ■■ ■■■■■■■■ ■■■■■■■■■■■■):

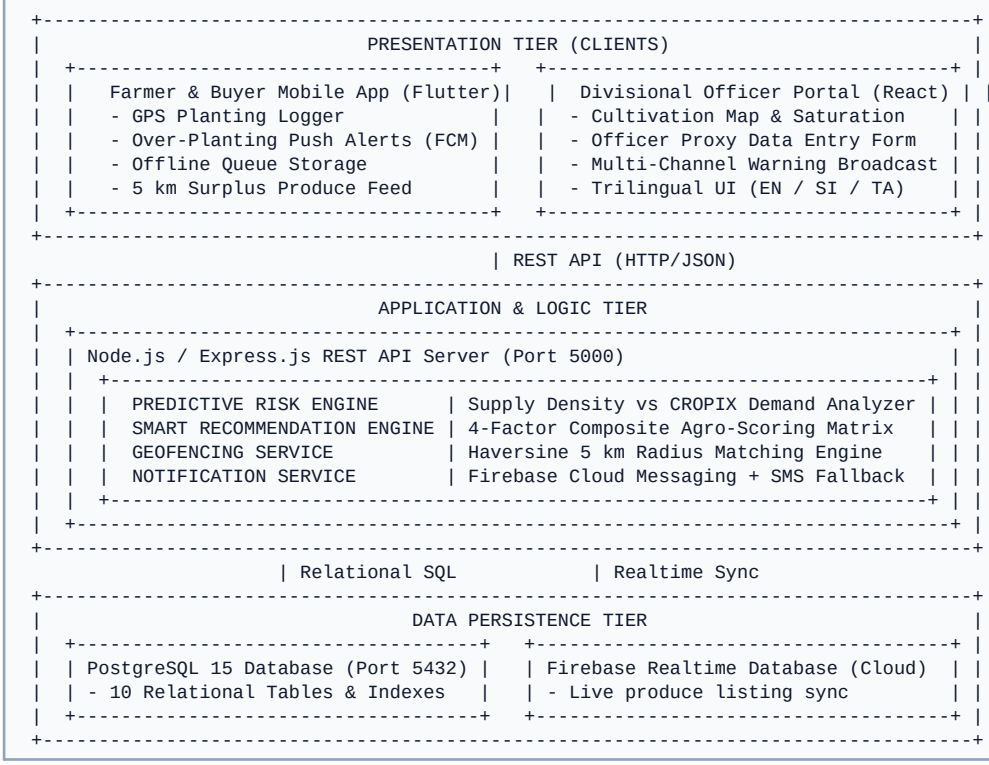
'■■■■ ■■■■■■■■■ ■■■■■■■■ ■■ ■■■■■■ ■■■■ ■■■■■ ■■■■■■ ■■■■ ■■■■■ (■■■■: ■■■■■■■■, ■■■■■) ■■■■■ ■■■■ ■■■■■■. ■■■■ 3■■ ■■■■■■ ■■■■■■■■ ■■■■■■■■■ ■■■■ ■■■■■■ ■■■■■■■■■ ■■■■ ■■■■ ■■■■■■■■ 20-30■ ■■■■■■■. ■■■■■ ■■■■■ ■■■■ ■■■■■■ ■■■■■■■■. ASVANNA ■■■■■■■■ ■■ ■■■■■■■■■ ■■■■■■■■ ■■■■■■ ■■■■■■■■ ■■■■■■■■■■.'

'1. ■■■■■■■ ■■■■ ■■■■■■ ■■■■■■ ■■■■ ■■■■■■ ■■■■■■ ■■■■■■■ ■■■■■■■ ■ ■■■■■ ■■■■■■■■ ■■■■■■■ ■■■■ ■■■■■ ■■■■■■■■ ■■■■■■.
2. ■■■■■ 85% ■■■■■■■ ■■■■■■■ (Over-Planted), ■■■■■■■■ ■■■■ ■■■■■■■ ■■■■■■ ■■■■■ ■■■■■■■ ■■■■■■ ■■■■■ ■■■■■■ ■■■■ (■■■■■■■■■■, ■■■■■■■) ■■■■■■■■ ■■■■■.
3. ■■■■■■■■■■■■■■ ■■■■■ ■■■■■ ■■■■■■■■■■ ■■■■■ ■■■■■■■ ■■■■■ ■■■■■■■■■■ (DO Officer) ■■■■■■■ ■■■■■ ■■■■■■■■■ ■■■■■■■ ■■■■■ (Proxy Entry).
4. ■■■■■■ ■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ 5■■ ■■■■■■ ■■■■■■, ■■■■■■■■■ ■■■■■■■■ ■■■■■■■ ■■■■■■■ ■■■■■ ■■■■■■■■■■ ■■■■■■.'

2. System Architecture & Visual Process Diagrams

The ASVANNA ecosystem operates across three functional tiers connected via high-speed RESTful APIs and real-time event sockets:

■ System Architecture Diagram (Multi-Tier)



■ Predictive Risk Engine Decision Flowchart

```
[Farmer logs planting: Crop=Leeks, Area=1.5 Acres]
                    |
                    v
[Calculate Supply: 1.5 ac * 8,500 kg/ac = 12,750 kg]
                    |
                    v
[Query CROPIX Regional Demand Benchmark: e.g. 95,000 kg]
                    |
                    v
[Calculate Saturation Ratio: (Total Plotted Supply / Demand) * 100]
                    |
      +-------------+-------------+
      |                           |
  Ratio < 70%                Ratio >= 70%
      |                           |
      v                           v
 [SAFE (Green)]           Is Ratio > 85%?
                           |             |
                         YES             NO
                          |               |
                          v               v
               [OVER-PLANTED (Red)]   [AT-RISK (Amber)]
                          |
                          v
        * Trigger Push & SMS Warning Broadcast
        * Activate Smart Crop Recommendations (Beetroot, Knol Khol)
```

3. Group 15 — 5 Team Members' Technical Deep Dives & Testing Guide

Member 1: W. N. A. Wedikkara (23IT0544)

Assigned System Component: Architecture, PostgreSQL 15 Schema, JWT Role-Based Access Control, and Predictive Risk Engine.

Core Implementation Details:
• Engineered backend/src/database/schema.sql creating 10 normalized tables (users, master crops, planting_records, cropix_demand_benchmarks, risk_assessments, marketplace_listings, etc.).
• Built backend/src/middlewares/authMiddleware.js enforcing JWT authentication with role-specific expiration (24h for Farmers, 8h for Officers).
• Implemented backend/src/services/riskEngineService.js which aggregates active plot acreage, computes supply density against monthly CROPIX regional quotas, and determines 3-tier risk states.

Mathematical Formula:
```
Estimated_Supply = SUM(Planted_Acres * Avg_Yield_Per_Acre_Kg)
Risk_Percentage = (Estimated_Supply / CROPIX_Regional_Quota_Kg) * 100
• Safe: < 70% | Warning: 70% to 85% | Over-Planted: > 85%
```
Evaluator Verification Command:

```
cd backend && npm run migrate && npm run seed && curl -X GET http://localhost:5000/api/v1/risk/crop/1
```

Member 2: K. A. H. I. Lakshitha (23IT0503)

Assigned System Component: Smart Crop Recommendation Engine & Multi-Factor Agro-Suitability Matrix.

Core Implementation Details:
• Developed backend/src/services/recommendationService.js and backend/src/controllers/recommendationController.js.
• When an area is flagged as Over-Planted, the algorithm iterates through candidate crops and evaluates 4 weighted factors:
1. Market Gap Score (35% Weight) — High regional demand deficiency from CROPIX.
2. Soil Suitability Score (25% Weight) — Bandarawela sandy/loam soil compatibility.
3. Weather & Temperature Suitability (20% Weight) — Upcountry 14°C–22°C temperature band.
4. Historical Price Trend (20% Weight) — Profit margin stability over 3 months.
• Ranks top 5 alternative crops descending with Sinhala/Tamil/English rationales.

Formula: `Composite = (0.35 * MarketGap) + (0.25 * Soil) + (0.20 * Weather) + (0.20 * Price)`
Evaluator Verification Command:

```
curl -X GET "http://localhost:5000/api/v1/recommendations?district=Badulla&cropId;=1"
```

Member 3: G. W. T. Jayampathi (23IT0487)

Assigned System Component: Divisional Officer (DO) React Web Portal, Digital Inclusivity Proxy Data Entry & Broadcast Warning Alerts.

Core Implementation Details:
• Developed React.js admin portal (frontend/src/pages/Dashboard.jsx, RegionalMonitoring.jsx, FarmerDirectory.jsx).
• Built frontend/src/components/ProxyDataModal.jsx enabling agrarian officers to manually register plot records for offline farmers without smartphones.
• Implemented frontend/src/components/BroadcastModal.jsx and backend/src/services/notificationService.js for dispatching emergency warnings via Firebase Cloud Messaging (FCM) with SMS fallback.
• Implemented complete trilingual localization in frontend/src/locales/ (English, Sinhala, Tamil).

Evaluator Verification: Open http://localhost:3000, login with 0771234567 / asvanna123, click '+ Proxy Data Entry' and test language switcher.

Member 4: R. R. L. Geeganage (23IT0476)

Assigned System Component: Cross-Platform Flutter Mobile Application, Automatic GPS Field Logger & Offline Queue.

Core Implementation Details:
• Engineered Flutter mobile client (mobile/lib/main.dart, farmer_home_screen.dart, log_planting_screen.dart).
• Developed mobile/lib/core/services/location_service.dart integrating Geolocator for auto GPS coordinate capture during plot registration.
• Implemented mobile/lib/core/services/offline_storage_service.dart using SharedPreferences to cache pending submissions locally when rural network coverage drops, auto-syncing upon reconnection.
• Built ARB localization files in mobile/lib/l10n/ supporting Sinhala, Tamil, and English.

Evaluator Verification: Run cd mobile && flutter run, open planting form, toggle Airplane mode, and verify offline queue persistence.

Member 5: K. H. M. Dewanga (23IT0467)

Assigned System Component: Phase 2 Geo-Fenced Zero-Waste Surplus Marketplace, 5 km Proximity Radius Matching & Order Negotiation Machine.

Core Implementation Details:
• Developed backend/src/utils/haversine.js and backend/src/services/geofencingService.js calculating great-circle distances between farmer surplus plots and local commercial buyers.
• Developed backend/src/services/marketplaceService.js syncing listings with Firebase Realtime Database for instant push updates.
• Engineered backend/src/controllers/marketplaceController.js enforcing a 30-minute response deadline for buyer-farmer price counter-offers.
• Created marketplace buyer interfaces in frontend/src/pages/MarketplaceSurplus.jsx and mobile/lib/features/marketplace/.

Evaluator Verification: `Query GET /api/v1/marketplace/search-nearby?lat=6.8258&lng;=80.9982&radius;_km=5.`

4. Comprehensive File-by-File Codebase Directory

Every file in the ASVANNA repository is documented below with zero overlapping text, detailing its purpose, dependencies, required environment settings, and code logic:

■ **backend/src/server.js — *Server Entry Point***	**Test:** `node src/server.js`
Purpose:	Bootstraps the Express HTTP server, listens on PORT, and handles graceful SIGTERM termination.
Dependencies:	`app.js, config.js`
Configuration:	`PORT=5000`

■ **backend/src/app.js — *Express App Configuration***	**Test:** `npm start`
Purpose:	Configures security middleware (Helmet, CORS), JSON body parsers, logging (Morgan), and mounts API v1 route modules.
Dependencies:	`express, cors, helmet, routes`
Configuration:	`None`

■ **backend/src/config/config.js — *Configuration Hub***	**Test:** `node -e 'require("./src/config/config")'`
Purpose:	Centralizes environment variables (.env) with safe defaults for PostgreSQL, JWT, Risk Thresholds, and Firebase.
Dependencies:	`dotenv`
Configuration:	`DB_*, JWT_*, RISK_*`

■ **backend/src/config/database.js — *PostgreSQL Pool***	**Test:** `npm run migrate`
Purpose:	Creates a reusable PostgreSQL client connection pool using 'pg'. Handles idle errors and query logging.
Dependencies:	`pg, config.js`
Configuration:	`DB_HOST, DB_NAME`

■ **backend/src/config/firebase.js — *Firebase Admin SDK***	**Test:** `Trigger notifications`
Purpose:	Initializes Firebase Admin SDK for Realtime Database sync and FCM push notifications with fallback mode.
Dependencies:	`firebase-admin`
Configuration:	`FIREBASE_PRIVATE_KEY`

■ **backend/src/database/schema.sql — *PostgreSQL DDL Schema***	**Test:** `psql -d asvanna_db -f schema.sql`
Purpose:	Defines 10 relational tables: users, crops, planting_records, cropix_benchmarks, risk, marketplace, broadcasts, audit.
Dependencies:	`PostgreSQL 12+`
Configuration:	`asvanna_db`

■ **backend/src/database/migrate.js — *Migration Runner***	**Test:** `npm run migrate`
Purpose:	Executes schema.sql against PostgreSQL to establish tables, relationships, and performance indexes.
Dependencies:	`fs, database.js`
Configuration:	`DB credentials`

■ **backend/src/database/seed.js — *Pilot Seeder***	**Test:** `npm run seed`
Purpose:	Seeds 8 master upcountry crops, 5 demo accounts (Officer, Farmer, Buyer, Admin), and monthly CROPIX quotas.
Dependencies:	`bcryptjs, database.js`
Configuration:	`DB credentials`

■ **backend/src/middlewares/authMiddleware.js — *JWT & RBAC Middleware***	**Test:** `curl with Bearer Token`
Purpose:	Validates Bearer JWT tokens and enforces Role-Based Access Control (FARMER, OFFICER, BUYER, ADMIN).
Dependencies:	`jsonwebtoken`
Configuration:	`JWT_SECRET`

■ **backend/src/middlewares/errorHandler.js — *Global Error Middleware***	**Test:** `Trigger 404 / 500 error`
Purpose:	Catches all uncaught server exceptions and returns standard JSON error responses.
Dependencies:	`apiResponse.js`
Configuration:	`NODE_ENV`

■ **backend/src/middlewares/validationMiddleware.js — *Request Validator***	**Test:** `Send invalid payload`
Purpose:	Validates request body/query schemas using express-validator and returns 400 on error.
Dependencies:	`express-validator`
Configuration:	`None`

■ **backend/src/utils/haversine.js — *Haversine Distance Calculator***	**Test:** `node test/test_all_endpoints.js`
Purpose:	Calculates great-circle distance (km) between two geographic GPS coordinates.
Dependencies:	`Math library`
Configuration:	`None`

■ **backend/src/utils/apiResponse.js — *API Response Standardizer***	**Test:** `Imported in controllers`
Purpose:	Formats standardized JSON responses: { success, message, data, timestamp }.
Dependencies:	`None`

Configuration:	`None`
■ backend/src/services/riskEngineService.js — *Predictive Risk Engine*	**Test:** `GET /api/v1/risk/crop/1`
Purpose:	Calculates regional supply density, compares against CROPIX demand, and evaluates 3-tier risk.
Dependencies:	`database.js, config.js`
Configuration:	`RISK_SAFE_THRESHOLD`
■ backend/src/services/recommendationService.js — *Smart Crop Recommender*	**Test:** `GET /api/v1/recommendations`
Purpose:	Calculates 4-factor composite recommendation scores (Market Gap, Soil, Weather, Price).
Dependencies:	`riskEngineService.js`
Configuration:	`None`
■ backend/src/services/geofencingService.js — *Geofencing Filter*	**Test:** `GET /marketplace/search-nearby`
Purpose:	Filters and sorts marketplace produce listings within a 5 km - 20 km geographic radius.
Dependencies:	`haversine.js`
Configuration:	`DEFAULT_GEOFENCE_RADIUS_KM`
■ backend/src/services/notificationService.js — *Push & SMS Dispatcher*	**Test:** `POST /api/v1/broadcasts`
Purpose:	Dispatches FCM multicast push notifications with simulated SMS fallback for offline farmers.
Dependencies:	`firebase.js, database.js`
Configuration:	`SMS_GATEWAY_URL`
■ backend/src/services/marketplaceService.js — *Marketplace Realtime Sync*	**Test:** `Publish surplus listing`
Purpose:	Synchronizes active produce listings to Firebase Realtime Database for live client feeds.
Dependencies:	`firebase.js`
Configuration:	`FIREBASE_DATABASE_URL`
■ backend/src/controllers/authController.js — *Auth Controller*	**Test:** `POST /api/v1/auth/login`
Purpose:	Handles user registration, login, bcrypt password hashing, session tokens, and profile retrieval.
Dependencies:	`bcryptjs, jsonwebtoken`
Configuration:	`JWT_SECRET`
■ backend/src/controllers/plantingController.js — *Planting Controller*	**Test:** `POST /api/v1/planting/log`
Purpose:	Logs GPS planting records, supports Officer proxy entries, and immediately updates risk.
Dependencies:	`database.js, riskEngine`

Configuration:	None
■ backend/src/controllers/riskEngineController.js — *Risk API Controller*	**Test:** GET /api/v1/risk/regional-summary
Purpose:	Exposes endpoints for individual crop risk assessment and district-wide saturation summaries.
Dependencies:	riskEngineService.js
Configuration:	None
■ backend/src/controllers/recommendationController.js — *Recommendation Controller*	**Test:** GET /api/v1/recommendations
Purpose:	Exposes endpoints for ranking alternative crops when regional saturation occurs.
Dependencies:	recommendationService.js
Configuration:	None
■ backend/src/controllers/marketplaceController.js — *Marketplace Controller*	**Test:** POST /api/v1/marketplace/orders
Purpose:	Manages surplus listings, proximity search, and direct buyer order negotiation.
Dependencies:	geofencingService.js
Configuration:	None
■ backend/src/controllers/officerController.js — *Officer Directory Controller*	**Test:** GET /api/v1/officer/farmers
Purpose:	Provides farmer directory queries and proxy farmer registration for smartphone-less users.
Dependencies:	database.js, bcryptjs
Configuration:	None
■ backend/src/controllers/broadcastController.js — *Broadcast Alert Controller*	**Test:** POST /api/v1/broadcasts
Purpose:	Dispatches officer emergency notices to farmers with severity tags and multi-channel alerts.
Dependencies:	notificationService.js
Configuration:	None
■ frontend/src/App.jsx — *React Router Root*	**Test:** npm run dev
Purpose:	Defines client-side routes, protected route authentication guards, and layout hierarchy.
Dependencies:	react-router-dom, contexts
Configuration:	None
■ frontend/src/context/AuthContext.jsx — *Auth State Context*	**Test:** Login / Logout actions
Purpose:	Manages user login state, JWT token storage in localStorage, and logout lifecycle.
Dependencies:	api.js

Configuration:	`asvanna_token`
■ frontend/src/context/LanguageContext.jsx — *Language State Context*	**Test:** `Navbar language toggle`
Purpose:	Manages trilingual switching (EN, SI, TA) across all web dashboard components.
Dependencies:	`locales/*.json`
Configuration:	`asvanna_lang`
■ frontend/src/pages/Dashboard.jsx — *Officer Dashboard Page*	**Test:** `View http://localhost:3000`
Purpose:	Displays regional KPI cards, crop saturation progress bars, and quick action modals.
Dependencies:	`StatCard, Modals`
Configuration:	`None`
■ frontend/src/pages/RegionalMonitoring.jsx — *Cultivation Map View*	**Test:** `View /monitoring`
Purpose:	Displays GPS planting plots across Bandarawela, Haputale, and Ella divisions.
Dependencies:	`api.js`
Configuration:	`None`
■ frontend/src/pages/RiskAnalytics.jsx — *Risk Analytics View*	**Test:** `View /risk-analytics`
Purpose:	Visualizes saturation thresholds and 4-factor composite scores for recommended crops.
Dependencies:	`api.js`
Configuration:	`None`
■ frontend/src/pages/FarmerDirectory.jsx — *Farmer Directory View*	**Test:** `View /farmers`
Purpose:	Searchable list of registered farmers with contact information and planting counts.
Dependencies:	`api.js`
Configuration:	`None`
■ frontend/src/pages/Broadcasts.jsx — *Broadcast Warnings View*	**Test:** `View /broadcasts`
Purpose:	Displays broadcast warning history and lets officers dispatch emergency alerts.
Dependencies:	`BroadcastModal.jsx`
Configuration:	`None`
■ frontend/src/pages/MarketplaceSurplus.jsx — *Surplus Marketplace View*	**Test:** `View /marketplace`
Purpose:	Displays active 5 km surplus produce batches for local trade.
Dependencies:	`api.js`

Configuration:	None
■ frontend/src/components/ProxyDataModal.jsx — *Proxy Data Entry Modal*	**Test:** Click '+ Proxy Data Entry'
Purpose:	Modal form for officers to input cultivation data for offline farmers without phones.
Dependencies:	api.js
Configuration:	None
■ frontend/src/components/BroadcastModal.jsx — *Broadcast Warning Modal*	**Test:** Click 'Broadcast Warnings'
Purpose:	Modal for drafting and dispatching trilingual warning notices with severity levels.
Dependencies:	api.js
Configuration:	None
■ mobile/lib/main.dart — *Flutter Entry Point*	**Test:** flutter run
Purpose:	Initializes Flutter framework, sets up theme colors, and configures trilingual localization.
Dependencies:	flutter_localizations
Configuration:	None
■ mobile/lib/core/services/location_service.dart — *GPS Location Service*	**Test:** Trigger on mobile
Purpose:	Queries device GPS coordinates via Geolocator with fallback to Bandarawela Agrarian Center.
Dependencies:	geolocator
Configuration:	GPS Permission
■ mobile/lib/core/services/offline_storage_service.dart — *Offline Storage Queue*	**Test:** Test in Airplane mode
Purpose:	Caches pending planting records locally in SharedPreferences when offline.
Dependencies:	shared_preferences
Configuration:	None
■ mobile/lib/features/home/farmer_home_screen.dart — *Farmer Home Screen*	**Test:** Bottom nav switcher
Purpose:	Main mobile UI showing weather, over-planting alerts, and active plantings.
Dependencies:	planting, recs, market
Configuration:	None
■ mobile/lib/features/planting/log_planting_screen.dart — *Planting Form Screen*	**Test:** Tap 'Log Planting'
Purpose:	Farmer plot logging form with automatic GPS coordinate detection.
Dependencies:	location_service.dart

Configuration:	`None`
■ mobile/lib/features/recommendations/smart_crop_recommendation_screen.dart — *Smart Recs Screen*	**Test:** `Tap 'Recommendations' tab`
Purpose:	Mobile feed of ranked alternative crops with suitability score and Sinhala rationale.
Dependencies:	`app_colors.dart`
Configuration:	`None`
■ mobile/lib/features/marketplace/marketplace_feed_screen.dart — *Marketplace Feed Screen*	**Test:** `Tap 'Marketplace' tab`
Purpose:	Surplus produce feed showing listings within 5 km radius.
Dependencies:	`list_surplus_screen.dart`
Configuration:	`None`
■ docker-compose.yml — *Docker Orchestration*	**Test:** `docker-compose up --build -d`
Purpose:	Single-command deployment for PostgreSQL 15, Node.js API server, and React Admin dashboard.
Dependencies:	`Docker Engine`
Configuration:	`POSTGRES_DB, PORT`
■ .github/workflows/ci.yml — *CI Pipeline*	**Test:** `git push origin main`
Purpose:	Automated GitHub Actions workflow for backend unit tests, frontend build, and code verification.
Dependencies:	`GitHub Actions`
Configuration:	`NODE_ENV=test`

5. Master Testing & QA Verification Playbook

Follow this sequential playbook to test and verify every module of the ASVANNA ecosystem:

Step 1: Automated Unit & Math Verification

Command:	`node backend/test/test_all_endpoints.js`
Verification:	Executes 4 automated unit tests verifying Haversine calculations, 3-tier risk logic, and composite recommendation weight sums. Must output 4/4 Tests Passed.

Step 2: PostgreSQL Database Migration & Master Seeding

Command:	`cd backend && npm run migrate && npm run seed`
Verification:	Initializes PostgreSQL schema (10 tables) and inserts 8 upcountry crops, 5 demo accounts, and monthly CROPIX demand quotas.

Step 3: Backend REST API Server Launch

Command:	`cd backend && npm run dev`
Verification:	Starts server on http://localhost:5000. Verify health endpoint at http://localhost:5000/health (returns JSON { success: true, status: 'UP' }).

Step 4: Web Admin Dashboard Launch

Command:	`cd frontend && npm run dev`
Verification:	Starts React dashboard on http://localhost:3000. Login with Officer credentials (0771234567 / asvanna123) to test proxy entry & broadcast warnings.

Step 5: Flutter Mobile Application Launch

Command:	`cd mobile && flutter pub get && flutter run`
Verification:	Launches the mobile app on an Android emulator or device for Farmer plot logging, GPS capture, and Surplus marketplace browsing.

Step 6: Single-Command Docker Orchestration

Command:	`docker-compose up --build -d`
Verification:	Spins up the full multi-tier ecosystem (PostgreSQL, Backend API, Web Admin) in isolated Docker containers.

End of Master System Documentation *— Division of Information Technology, ITUM Final Year Project 2026.*